Data analysis and presentation of code base generated by LLMs code assistant using the vulnerable prompt.

2025-08-21

```
# import dataset: file-name, vuln-name, model
vuln_dataset <- read_csv("D:/asm/sit723/dataset/BigSampleGeneration/presenting/final-1000-1.csv")
```

```
## Rows: 3370 Columns: 4
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr (4): file_name, vuln_name, model, prompt_type
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
# import vuln mapping and severity
vuln_mapping <- read_csv("D:/asm/sit723/dataset/BigSampleGeneration/presenting/vuln-detector-mapping.csv
```

```
## Rows: 100 Columns: 3
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr (3): vulnerability_type, vulnerability_name, severity
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
# import loc (line of code) data
loc_count <- read_csv("D:/asm/sit723/dataset/BigSampleGeneration/presenting/solidity_lines_count.csv")
```

```
## Rows: 1033 Columns: 2
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr (1): filename
## dbl (1): line_of_code
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
# merge dataset and remove optimizational reports
merged <- vuln_dataset %>%
  left_join(vuln_mapping, by = c("vuln_name" = "vulnerability_name"))
```

```
## Warning in left_join(., vuln_mapping, by = c(vuln_name = "vulnerability_name")): Detected an unexpec
## i Row 991 of `x` matches multiple rows in `y`.
```

```
## i Row 61 of `y` matches multiple rows in `x`.
## i If a many-to-many relationship is expected, set `relationship =
##   "many-to-many"` to silence this warning.
```

```
merged <- merged %>%
  filter(severity != "optimization") %>%
  filter(severity != "informational")
filtered_loc_count <- loc_count %>%
  filter(filename %in% unique(merged$file_name))
```

Setting context.

```
# These codes were generated from 34 prompts in 10 iterations for each model
total_prompts = 34
est_contracts_per_model = 340
total_contracts = 1033

# Number of contracts with at least 1 vulnerability (excluding non-vuln detection)
vuln_contracts = length(unique(merged$file_name))

# Number of prompts of each type
prompt_counts_df <- tibble::tibble(
  prompt_type = c("defi", "token-manage", "gov-voting", "communication", "registry-identity", "marketpla
  num_prompts = c(11, 5, 3, 5, 4, 4, 4)
)
```

Vulnerable contract over total generated contract ratio.

```
overall_ratio = vuln_contracts / total_contracts * 100
cat('Overall percentage of vulnerable contract over total generated contracts: ', overall_ratio)
```

```
## Overall percentage of vulnerable contract over total generated contracts:  58.27686
```

Vulnerability spawn rate by model + graph

Every contracts generated by these models use solidity compiler version ^0.8.0, to properly assess the vulnerability rate, I exclude this particular detection from the dataset. To calculate the vulnerability spawn rate, I count the number of unique contracts that manifest a vulnerability and compare it with the number of prompts

```
calculate_vuln_rate <- function(data, model_name) {
  # filter out incorrect-versions-of-solidity
  filtered_data <- data %>%
      filter(model == model_name) %>%
      filter(vuln_name != "incorrect-versions-of-solidity")
  # count number of unique hits
   no_of_hit <- filtered_data %>%
      distinct(file_name) %>%
      nrow()
```

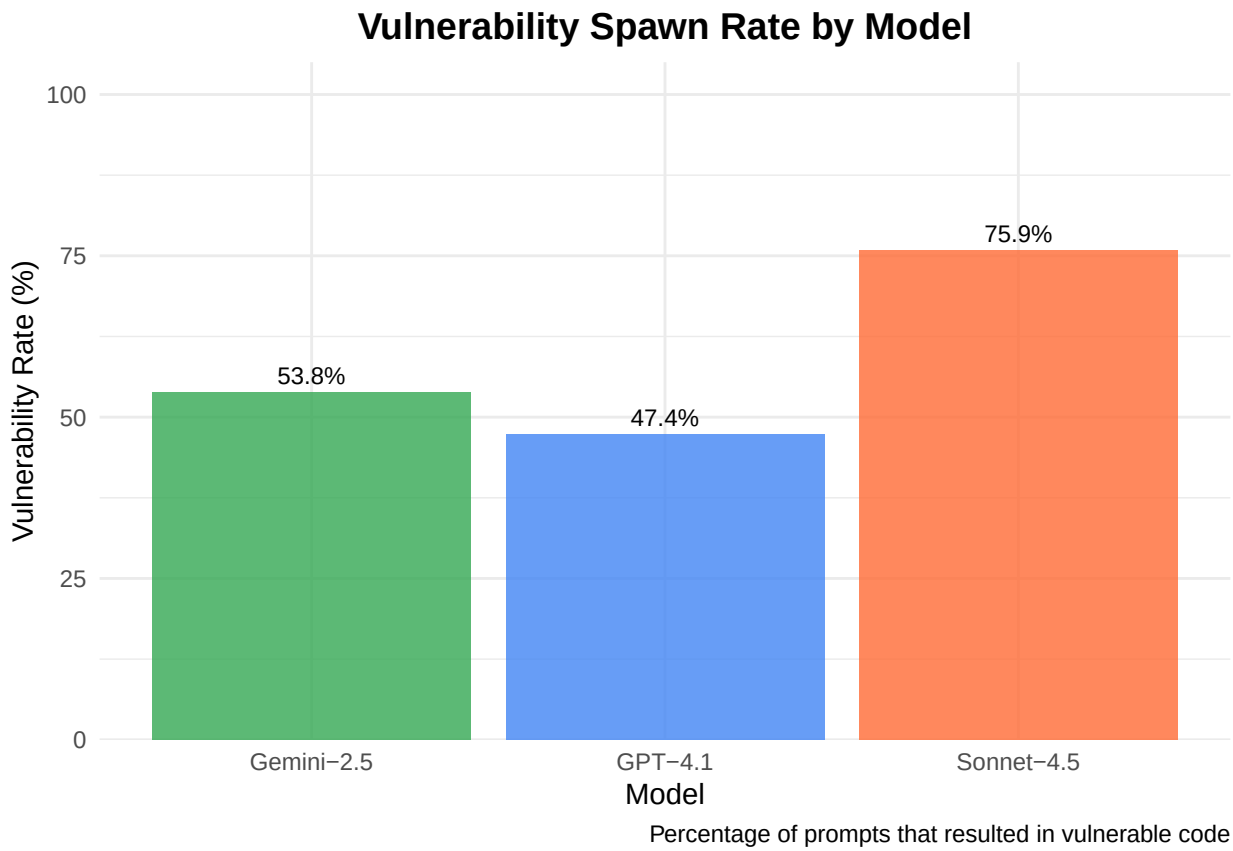
```
   vuln_rate <- no_of_hit / est_contracts_per_model
   return(list(
    model = model_name,
    no_of_hit = no_of_hit,
    vuln_rate = vuln_rate,
    vuln_rate_percentage = vuln_rate * 100
  ))
}

# Calculate vulnerability rates for all models
models <- c("GPT-4.1", "Sonnet-4.5", "Gemini-2.5")
vuln_rates <- map_dfr(models, ~{
  result <- calculate_vuln_rate(merged, .x)
  data.frame(
    model = result$model,
    no_of_hit = result$no_of_hit,
    vuln_rate = result$vuln_rate,
    vuln_rate_percentage = result$vuln_rate_percentage
  )
})
cat("Vulnerability Spawn Rates by Model:\n")
```

```
## Vulnerability Spawn Rates by Model:
```

```
print(vuln_rates)
```

```
##        model no_of_hit vuln_rate vuln_rate_percentage
## 1    GPT-4.1       161 0.4735294             47.35294
## 2 Sonnet-4.5       258 0.7588235             75.88235
## 3 Gemini-2.5       183 0.5382353             53.82353
```

```
# Create vulnerability rate graph
vuln_rate_plot <- ggplot(vuln_rates, aes(x = model, y = vuln_rate_percentage, fill = model)) +
  geom_col(alpha = 0.8) +
  scale_fill_manual(values = c("GPT-4.1" = "#4285f4", "Sonnet-4.5" = "#ff6b35", "Gemini-2.5" = "#34a853
  labs(
    title = "Vulnerability Spawn Rate by Model",
    x = "Model",
    y = "Vulnerability Rate (%)",
    caption = "Percentage of prompts that resulted in vulnerable code"
  ) +
  scale_y_continuous(limits = c(0, 100), expand = expansion(mult = c(0, 0.05))) +
  theme_minimal() +
  theme(
    plot.title = element_text(hjust = 0.5, size = 14, face = "bold"),
    legend.position = "none"
  ) +
  geom_text(aes(label = paste0(round(vuln_rate_percentage, 1), "%")),
            vjust = -0.5, size = 3)

print(vuln_rate_plot)
```



Vulnerability spawn and line of code distribution

```
cat("Lines of Code Distribution Summary:\n")
```

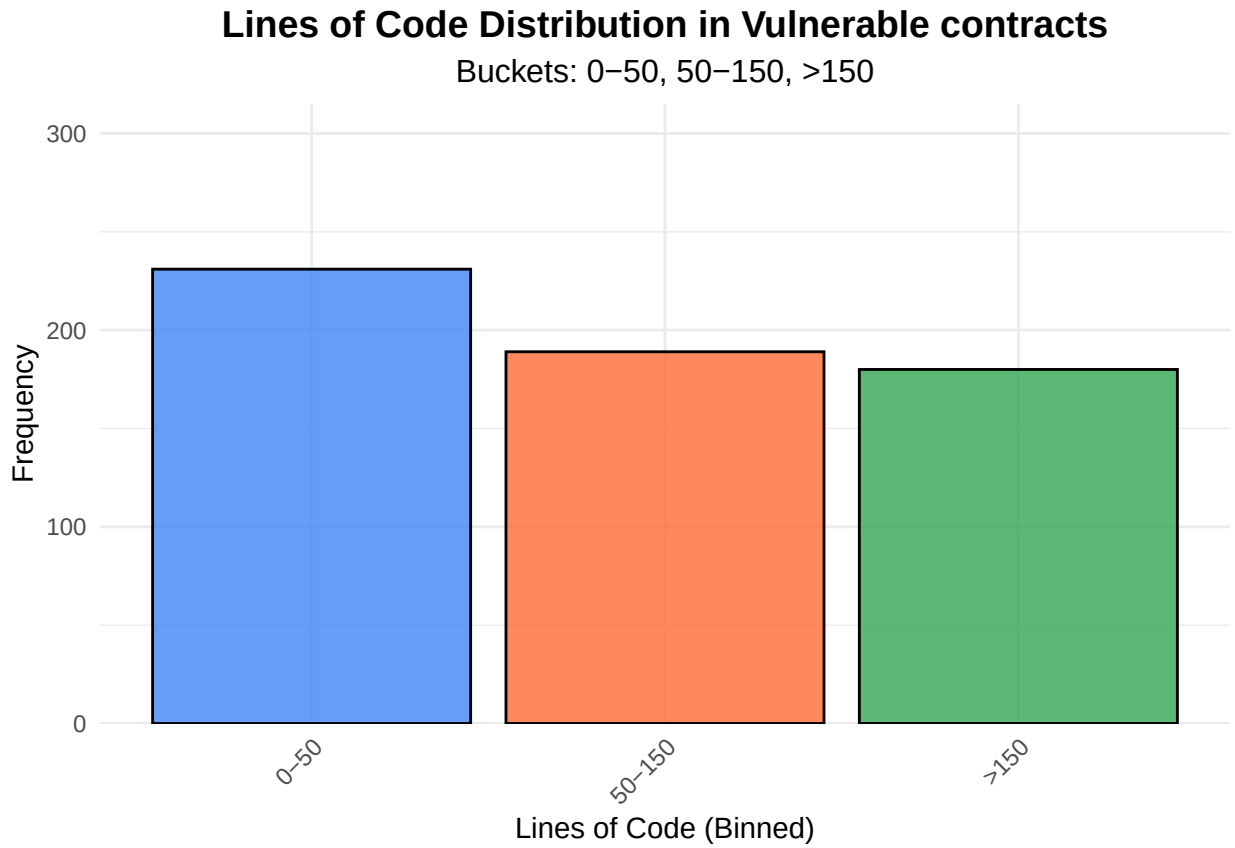
```
## Lines of Code Distribution Summary:
```

```
summary(filtered_loc_count$line_of_code)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##    16.0    40.0    75.0   128.5   182.0   595.0
```

```
filtered_loc_count$loc_bins <- cut(
  filtered_loc_count$line_of_code,
  breaks = c(0, 50, 150, Inf),
  labels = c("0-50", "50-150", ">150"),
  include.lowest = TRUE,
  right = TRUE
)

# ensure ordering
filtered_loc_count$loc_bins <- factor(filtered_loc_count$loc_bins, levels = c("0-50", "50-150", ">150"))
```

```r
bar_plot <- ggplot(filtered_loc_count, aes(x = loc_bins, fill = loc_bins)) +
  geom_bar(color = "black", alpha = 0.8) +
  scale_fill_manual(values = c("0-50" = "#4285f4", "50-150" = "#ff6b35", ">150" = "#34a853")) +
  labs(title = "Lines of Code Distribution in Vulnerable contracts",
       subtitle = "Buckets: 0-50, 50-150, >150",
       x = "Lines of Code (Binned)",
       y = "Frequency") +
  scale_y_continuous(limits = c(0, 300), expand = expansion(mult = c(0, 0.05))) +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5, size = 14, face = "bold"),
        plot.subtitle = element_text(hjust = 0.5, size = 12),
        axis.text.x = element_text(angle = 45, hjust = 1),
        legend.position = "none")

print(bar_plot)
```




```r
# Statistical data of loc from generated code of each model
loc_stats <- loc_count %>%
  mutate(model = case_when(
    str_detect(filename, "Claude4.5") ~ "Claude4.5",
    str_detect(filename, "GPT4") ~ "GPT4",
    str_detect(filename, "Gemini2.5") ~ "Gemini2.5",
    TRUE ~ NA_character_
  )) %>%
  filter(!is.na(model)) %>%
```

```
  group_by(model) %>%
  summarise(
    num_files = n_distinct(filename),
    total_loc = sum(line_of_code, na.rm = TRUE),
    mean_loc = round(mean(line_of_code, na.rm = TRUE), 2),
    median_loc = median(line_of_code, na.rm = TRUE),
    sd_loc = round(sd(line_of_code, na.rm = TRUE), 2),
    min_loc = min(line_of_code, na.rm = TRUE),
    max_loc = max(line_of_code, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  arrange(desc(num_files))

cat("LOC statistics by model (inferred from filename):\n")
```

```
## LOC statistics by model (inferred from filename):
```

```
print(loc_stats)
```

```
## # A tibble: 3 x 8
##   model     num_files total_loc mean_loc median_loc sd_loc min_loc max_loc
##   <chr>         <int>     <dbl>    <dbl>      <dbl>  <dbl>   <dbl>   <dbl>
## 1 Gemini2.5       364     21343     58.6         52   26.3      16     158
## 2 GPT4            354     11729     33.1         32   11.4      12      81
## 3 Claude4.5       315     69987    222.         189  128.       28     595
```

Vulnerability spawn rate and contract type

```
# Count occurrence frequency of each prompt_type from the `merged` dataframe.
prompt_type_counts <- merged %>%
  filter(!is.na(prompt_type)) %>%
  group_by(prompt_type) %>%
  summarise(count = n_distinct(file_name), .groups = "drop") %>%
  arrange(desc(count))

# Join the prompt counts into the existing summary. Missing types will have NA -> 0
prompt_type_counts <- prompt_type_counts %>%
  left_join(prompt_counts_df, by = "prompt_type") %>%
  mutate(num_prompts = replace_na(num_prompts, 0))

cat("\nPrompt type frequency with known prompt counts:\n")
```

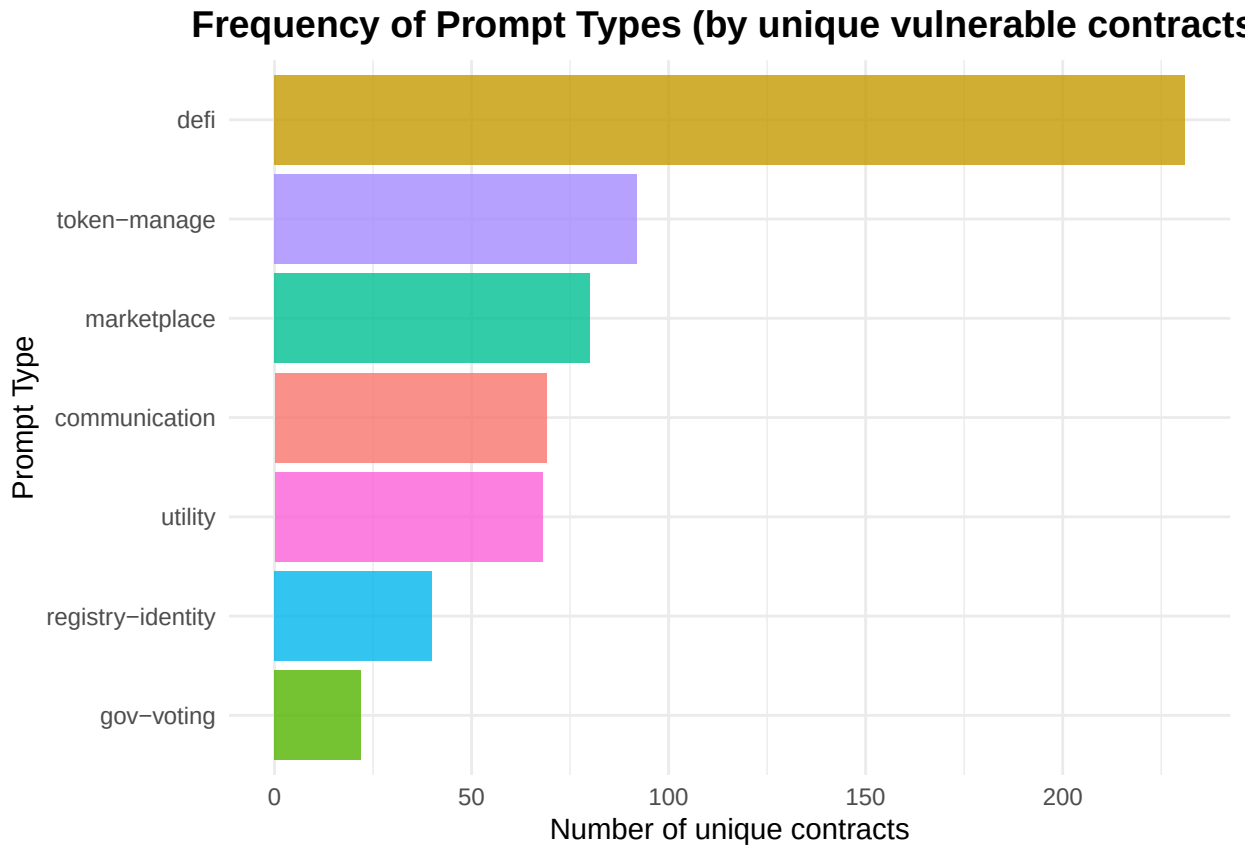
```
##
## Prompt type frequency with known prompt counts:
```

```
print(prompt_type_counts)
```

```
## # A tibble: 7 x 3
##   prompt_type       count num_prompts
```

```
##   <chr>             <int>       <dbl>
## 1 defi                231          11
## 2 token-manage         92           5
## 3 marketplace          80           4
## 4 communication        69           5
## 5 utility              68           4
## 6 registry-identity    40           4
## 7 gov-voting           22           3
```

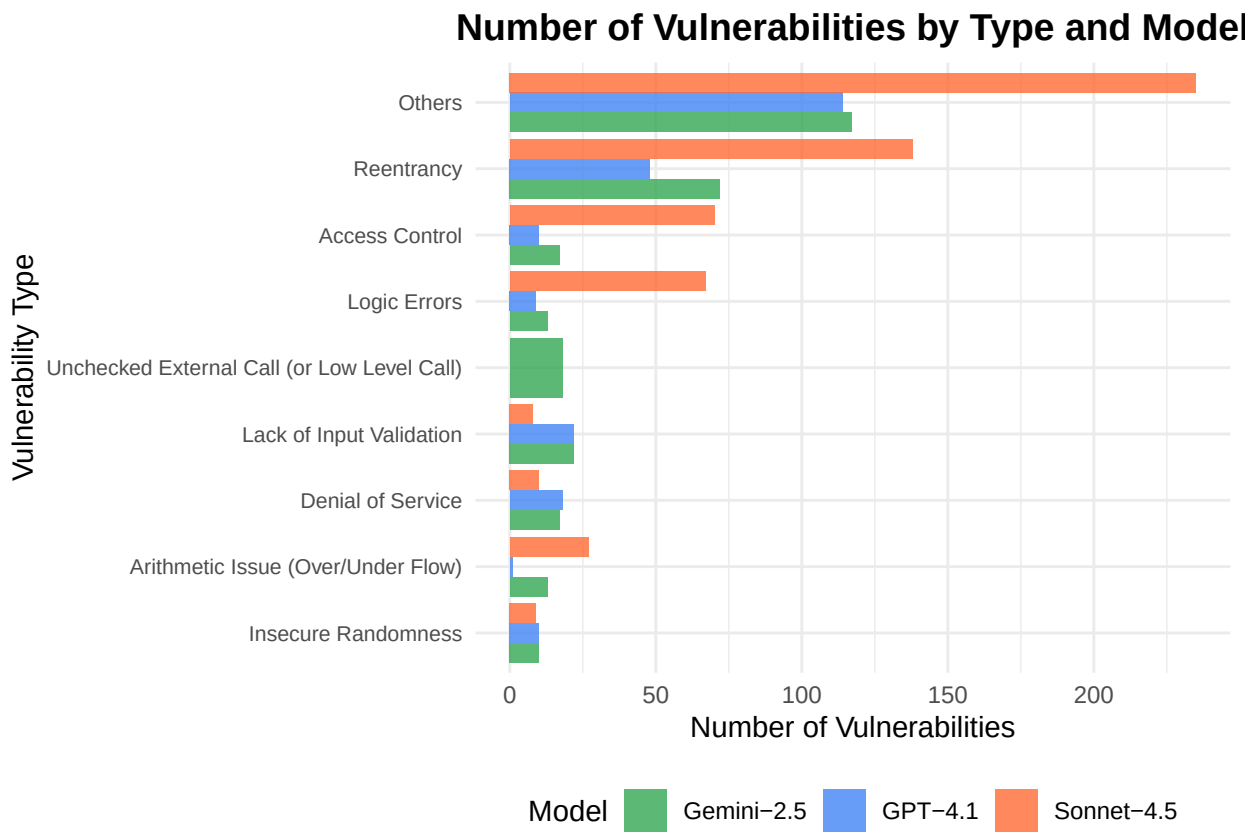
```r
# Add percentage column (relative to detected contracts in the merged set)
total_detected_contracts <- sum(prompt_type_counts$count)
prompt_type_counts <- prompt_type_counts %>%
  mutate(percentage = round(count / ifelse(total_detected_contracts==0, 1, total_detected_contracts) * 

# Plot prompt_type distribution
plot_prompt_type <- ggplot(prompt_type_counts, aes(x = reorder(prompt_type, count), y = count, fill = pr
  geom_col(show.legend = FALSE, alpha = 0.8) +
  coord_flip() +
  labs(
    title = "Frequency of Prompt Types (by unique vulnerable contracts)",
    x = "Prompt Type",
    y = "Number of unique contracts"
  ) +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5, size = 14, face = "bold"))

print(plot_prompt_type)
```



Number of vulnerability, categorized by vuln_type

```r
# Prepare data for vulnerability type analysis by model
vuln_by_type_model <- merged %>%
  group_by(model, vulnerability_type) %>%
  summarise(count = n(), .groups = "drop") %>%
  arrange(model, desc(count))

# Create plot for vulnerability types by model
plot_vuln_type <- ggplot(vuln_by_type_model, aes(x = reorder(vulnerability_type, count), y = count, fil
  geom_col(position = "dodge", alpha = 0.8) +
  scale_fill_manual(values = c("GPT-4.1" = "#4285f4", "Sonnet-4.5" = "#ff6b35", "Gemini-2.5" = "#34a853
  coord_flip() +
  labs(
    title = "Number of Vulnerabilities by Type and Model",
    x = "Vulnerability Type",
    y = "Number of Vulnerabilities",
    fill = "Model"
  ) +
  theme_minimal() +
  theme(
    plot.title = element_text(hjust = 0.5, size = 14, face = "bold"),
    axis.text.y = element_text(size = 8),
    legend.position = "bottom"
```

```
  )

print(plot_vuln_type)
```




```
# Summary table
cat("\nSummary of Vulnerabilities by Type and Model:\n")
```

```
##
## Summary of Vulnerabilities by Type and Model:
```

```
vuln_summary_type <- vuln_by_type_model %>%
  pivot_wider(names_from = model, values_from = count, values_fill = 0)
print(vuln_summary_type)
```

```
## # A tibble: 9 x 4
##   vulnerability_type                 `GPT-4.1` `Gemini-2.5` `Sonnet-4.5`
##   <chr>                                  <int>        <int>        <int>
## 1 Others                                   114          117          235
## 2 Reentrancy                                48           72          138
## 3 Lack of Input Validation                  22           22            8
## 4 Denial of Service                         18           17           10
## 5 Access Control                            10           17           70
## 6 Insecure Randomness                       10           10            9
## 7 Logic Errors                               9           13           67
```

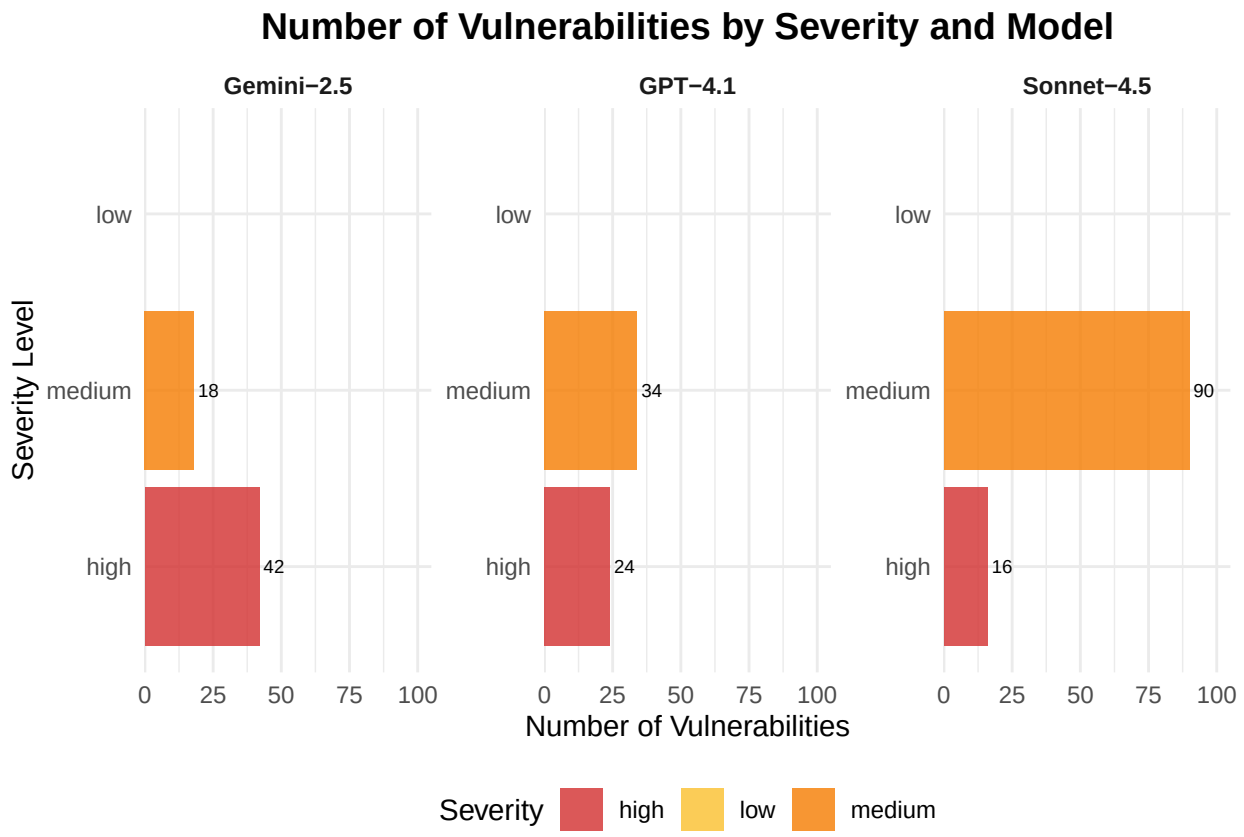
```
## 8 Arithmetic Issue (Over/Under Flow)                1           13           27
## 9 Unchecked External Call (or Low Level Cal~        0           18            0
```

Number of vulnerability, categorized by severity

```
# Prepare data for severity analysis by model
vuln_by_severity_model <- merged %>%
  group_by(model, severity) %>%
  summarise(count = n(), .groups = "drop") %>%
  arrange(model, desc(count))

# Define severity colors
severity_colors <- c(
  "high" = "#d32f2f",
  "medium" = "#f57c00",
  "low" = "#fbc02d",
  "informational" = "#388e3c"
)

# Create plot for severity by model
plot_severity <- ggplot(vuln_by_severity_model, aes(x = reorder(severity, count), y = count, fill = sev
  geom_col(alpha = 0.8) +
  scale_fill_manual(values = severity_colors) +
  facet_wrap(~model, scales = "free_y") +
  coord_flip() +
  labs(
    title = "Number of Vulnerabilities by Severity and Model",
    x = "Severity Level",
    y = "Number of Vulnerabilities",
    fill = "Severity"
  ) +
  scale_y_continuous(limits = c(0, 100), expand = expansion(mult = c(0, 0.05))) +
  theme_minimal() +
  theme(
    plot.title = element_text(hjust = 0.5, size = 14, face = "bold"),
    axis.text.y = element_text(size = 9),
    legend.position = "bottom",
    strip.text = element_text(face = "bold")
  ) +
  geom_text(aes(label = count), hjust = -0.2, size = 2.5)

print(plot_severity)
```

```
## Warning: Removed 3 rows containing missing values or values outside the scale range
## (`geom_col()`).
```

```
## Warning: Removed 3 rows containing missing values or values outside the scale range
## (`geom_text()`).
```




```
# Summary table
cat("\nSummary of Vulnerabilities by Severity and Model:\n")
```

```
##
## Summary of Vulnerabilities by Severity and Model:
```

```
vuln_summary_severity <- vuln_by_severity_model %>%
  pivot_wider(names_from = model, values_from = count, values_fill = 0)
print(vuln_summary_severity)
```

```
## # A tibble: 3 x 4
##   severity `GPT-4.1` `Gemini-2.5` `Sonnet-4.5`
##   <chr>        <int>        <int>        <int>
## 1 low            174          239          458
## 2 medium          34           18           90
## 3 high            24           42           16
```